

PROSJEKT BALLTRACK

Denne rapporten skal forklare oppbygging og struktur på en gjenkjennelig måte for enklere overtakelse av prosjektet.

Hand-over
rapport

Innhold

1.0 Innledning	2
1.1 Hva systemet gjør	3
1.2 Hvordan systemet er bygget opp	4
1.3 Ansvarsdeling i systemet	5
1.4 Reguleringsprinsipp og systemflyt	6
1.5 Grensesnitt og API-er	8
1.6 HMI og brukergrensesnitt.....	9
1.7 Simulering og testing uten hardware.....	10
1.8 Systemets driftsmodi og tilstander	11
1.9 Sammenheng med klassisk PLS-arkitektur	12
1.10 Verktøy for utvikling og testing	13
1.10.1 Versjonskontroll med Git og GitHub.....	14
1.10.2 Python-miljø og avhengigheter	14
1.10.3 Utviklingsmiljø (VS Code).....	14
1.10.4 Terminalbruk på Windows og Linux.....	15
1.11 Hvordan orientere seg i prosjektet	15
1.12 Hvordan jobbe videre med prosjektet	18
1.13 Begrensninger og forutsetninger.....	19
1.14 Videre bruk som handover	20
1.15 Ordliste	21

1.0 Innledning

Denne rapporten dokumenterer systemarkitektur, virkemåte og utviklingsrammer for prosjektet **Balltrack (V8)**. Dokumentet er utformet som et hand-over-dokument og har som mål å gi en helhetlig og lett tilgjengelig innføring i systemet, slik at nye studenter, faglærere eller utviklere raskt kan sette seg inn i prosjektet og videreføre arbeidet på en strukturert og forsvarlig måte.

Balltrack er utviklet som et lærings- og demonstrasjonssystem innen automasjon og regulering. Systemet kombinerer en fysisk prosess med reguleringsalgoritmer, programvarearkitektur og grafisk brukergrensesnitt i én sammenhengende løsning. Arkitekturen er bevisst bygget med tydelig separasjon mellom hardware, styringslogikk og HMI, i tråd med prinsipper brukt i industriell automasjon og PLS-baserte systemer. Denne strukturen gir både god systemforståelse og høy overføringsverdi til industrielle anvendelser.

Rapporten er ikke ment som en linje-for-linje-gjennomgang av kildekoden. Fokus ligger i stedet på systemforståelse, arkitektur, ansvarsdeling og samspill mellom systemets komponenter. Hensikten er å forklare hvordan systemet er bygget opp, hvordan det er ment å brukes, og hvorfor sentrale arkitektoniske valg er gjort. Dette gjør dokumentet relevant også for lesere som ikke har inngående kjennskap til Python, men som har bakgrunn innen automasjon og regulering.

Målgruppen for rapporten er primært automasjonsstudenter som skal bruke eller videreutvikle Balltrack i undervisning, øving eller prosjektarbeid. Samtidig er dokumentet utformet slik at det også kan fungere som et teknisk hand-over-dokument i en mer profesjonell sammenheng, der systemet overleveres til nye utviklere eller vurderes som grunnlag for videre arbeid.

For å støtte dette formålet er rapporten strukturert trinnvis. Den starter med en forklaring av hva systemet gjør og hvordan det er bygget opp, før den går videre til reguleringsprinsipper, grensesnitt og brukerinteraksjon. Videre beskrives simulering og testing uten fysisk hardware, sammenhengen med klassisk PLS-arkitektur, samt verktøy brukt i utvikling og testing. Avslutningsvis gis veiledning for videre arbeid og en oversikt over systemets begrensninger og forutsetninger.

Målet er at leseren, etter å ha gjennomgått rapporten, skal ha tilstrekkelig forståelse til å kunne forklare systemets oppbygning, gjøre endringer på riktig nivå i arkitekturen og videreutvikle prosjektet uten å bryte de grunnleggende prinsippene det er bygget på.

1.1 Hva systemet gjør

Balltrack er et reguleringsystem som har som oppgave å styre posisjonen til en ball langs en horisontal bane. Systemets hovedfunksjon er å holde ballen på en ønsket posisjon, kalt settpunkt, ved kontinuerlig å måle ballens faktiske posisjon og justere en servoaktuert mekanisme som påvirker banens helning. Systemet utgjør dermed et klassisk eksempel på et lukket reguleringsystem, slik det behandles innen automasjon og reguleringssteknikk.

Den regulerte prosessen kan beskrives som følger: Ballens posisjon langs banen representerer prosessverdien (PV), mens brukerens ønskede ballposisjon representerer settpunktet (SP). Systemet beregner kontinuerlig avviket mellom disse to størrelsene, og dette avviket brukes som inngang til regulatoren. Regulatorens utgang benyttes deretter til å styre servoens posisjon, som igjen påvirker ballens bevegelse langs banen. Denne reguleringsløyfen kjøres kontinuerlig så lenge systemet er aktivt.

Systemet er bygget opp rundt normaliserte signaler. Ballens posisjon uttrykkes som en verdi mellom 0 og 1, der 0 representerer den ene enden av banen og 1 representerer den andre. Tilsvarende benyttes normaliserte verdier for settpunkt og regulatorutgang. Denne tilnærmingen gjør systemet enklere å forstå, mer fleksibelt ved endringer i fysisk oppsett og mindre avhengig av spesifikke hardware-parametere.

Balltrack kan operere i to hovedmodi: med fysisk hardware eller i simulert modus. I hardware-modus leses ballens posisjon fra en analog sensor via en analog-til-digital-omformer, og servoen styres ved hjelp av en ekstern PWM-kontroller. I simulert modus erstattes den fysiske prosessen av en programvaremodell som genererer realistiske prosessverdier. For resten av systemet, inkludert kontroller og HMI, fremstår disse to modusene som funksjonelt like, noe som er et bevisst arkitektonisk valg.

Et sentralt mål med Balltrack er ikke bare å levere stabil regulering, men å gjøre reguleringsprosessen synlig og forståelig. Systemet er derfor bygget slik at sentrale signaler som settpunkt, prosessverdi, regulatorutgang og bidragene fra de enkelte regulatorleddene er tilgjengelige for visning i brukergrensesnittet. Dette gir brukeren mulighet til å observere hvordan endringer i parametere eller settpunkt påvirker systemets dynamikk i sanntid.

Oppsummert kan Balltrack beskrives som et komplett, men avgrenset reguleringsystem som demonstrerer grunnleggende prinsipper innen automasjon: måling, regulering, aktivering og overvåking. Systemet er enkelt nok til å forstå i sin helhet, samtidig som det er strukturert og realistisk nok til å gi erfaring som er direkte overførbart til industrielle automasjons- og PLS-baserte systemer.

1.2 Hvordan systemet er bygget opp

Balltrack er bygget opp som et lagdelt system der hver del har et tydelig definert ansvar. Denne strukturen er valgt for å gjøre systemet forståelig, robust og lett å videreutvikle, og følger de samme prinsippene som benyttes i industrielle automasjonssystemer basert på PLS og HMI. Arkitekturen er bevisst utformet slik at endringer i en del av systemet i minst mulig grad påvirker andre deler.

Nederst i systemet finnes hardware-laget. Dette laget har ansvar for all direkte kommunikasjon med fysiske komponenter, som sensorer og aktuatorer. I Balltrack betyr dette lesing av analog posisjonsinformasjon fra en analog-til-digital-omformer og generering av pulsbreddemodulerte signaler til servoen via en ekstern PWM-kontroller. Hardware-laget inneholder ingen reguleringslogikk og ingen kunnskap om hvordan signalene brukes videre i systemet. Det har kun ansvar for å levere rå måleverdier og utføre kommandoer pålitelig.

Over hardware-laget ligger kontroller- og reguleringslaget. Dette laget utgjør kjernen i systemet og kan direkte sammenlignes med programmet som kjører i en PLS. Her håndteres lesing av prosessverdi, beregning av avvik, utførelse av reguleringsalgoritmen og generering av styresignal til aktuatoren. Kontrolleren holder også oversikt over systemets interne tilstand, som for eksempel om reguleringen er aktiv, om systemet er i manuell modus, og hvilke verdier som sist ble beregnet. All reguleringslogikk er samlet i dette laget for å sikre tydelig ansvarsdeling.

Mellom kontrolleren og brukergrensesnittet finnes et eksplisitt grensesnittlag. Dette laget fungerer som et bindeledd som eksponerer kontrollerens funksjonalitet på en kontrollert og strukturert måte. I stedet for at HMI-et kommuniserer direkte med kontrollerens interne variabler og metoder, går all kommunikasjon via et definert sett med funksjoner og verdier. Denne løsningen gjør systemet mer robust og tilsvarer bruken av datablocker eller UDT-er mellom PLS og HMI i industrielle løsninger.

Øverst i arkitekturen ligger HMI-laget. Dette laget består av grafiske komponenter som presenterer systemets tilstand og gir brukeren mulighet til å styre systemet. HMI-et har ingen direkte kunnskap om hvordan reguleringen er implementert eller hvordan hardware fungerer. Det forholder seg kun til de grensesnittene som er gjort tilgjengelige gjennom grensesnittlaget. Dette gjør det mulig å endre både reguleringslogikk og hardware uten å måtte endre brukergrensesnittet, så lenge grensesnittene forblir stabile.

Et sentralt designvalg i Balltrack er at samme arkitektur benyttes både når systemet kjøres med fysisk hardware og når det kjøres i simulert modus. Dette betyr at kontroller, grensesnitt og HMI er identiske i begge tilfeller. Forskjellen ligger kun i hvordan prosessverdier genereres og hvordan styresignaler håndteres. Dette gir en ryddig og konsistent arkitektur der simulering ikke er et tillegg, men en integrert del av systemet.

Samlet sett er Balltrack bygget opp som et modulært og lagdelt system der hver del har et klart ansvar og tydelige grensesnitt mot resten av systemet. Denne oppbygningen gjør det mulig å forstå systemet som en helhet, samtidig som hver del kan studeres og videreutvikles isolert. For automasjonsstudenter gir dette en arkitektur som er direkte gjenkjennelig fra PLS-baserte systemer, selv om implementasjonen er gjort i et moderne programmeringsspråk.

1.3 Ansvarsdeling i systemet

En av de viktigste egenskapene ved Balltrack er den tydelige ansvarsdelingen mellom systemets ulike deler. Denne ansvarsdelingen er ikke tilfeldig, men et bevisst arkitektonisk valg som følger etablerte prinsipper fra industriell automasjon. Målet er å gjøre systemet forståelig, robust og enkelt å videreutvikle uten at endringer i en del skaper uforutsette konsekvenser i andre deler.

Hardware-laget har ansvar for all direkte kommunikasjon med fysiske komponenter. Dette innebærer lesing av rå måleverdier fra sensorer og utsending av styresignaler til aktuatorer. Hardware-laget kjenner verken til reguleringsalgoritmer, settpunkt eller brukergrensesnitt. Det leverer kun data og utfører kommandoer. På samme måte som IO-moduler i et PLS-system er hardware-laget i Balltrack ment å være så enkelt og forutsigbart som mulig.

Kontroller- og reguleringslaget har ansvar for all logikk knyttet til systemets oppførsel. Dette laget avgjør hvordan systemet reagerer på endringer i prosessverdi og settpunkt, hvordan reguleringen utføres, og hvordan ulike driftsmodi håndteres. Kontrolleren fungerer som systemets beslutningstaker og tilsvarende PLS-programmet i et industrielt anlegg. All reguleringslogikk, tilstandslogikk og beregning av styresignaler er samlet her. Dette sikrer at endringer i reguleringsstrategi eller algoritmer kan gjøres uten å påvirke hardware eller brukergrensesnitt direkte.

Grensesnittlaget mellom kontroller og HMI har ansvar for å definere hvordan informasjon og kommandoer utveksles. Dette laget fungerer som en kontrakt mellom styringslogikken og brukergrensesnittet. HMI-et får kun tilgang til et begrenset og kontrollert sett med

funksjoner og data, og kan ikke manipulere kontrollerens interne tilstand direkte. Denne tilnærmingen er direkte sammenlignbar med bruk av datablocker eller strukturerte variabler mellom PLS og HMI i industrielle systemer, og bidrar til forutsigbarhet og stabilitet.

HMI-laget har ansvar for presentasjon og brukerinteraksjon. Dette inkluderer visning av prosessverdi, settpunkt, regulatorutgang og systemstatus, samt mottak av brukerkommandoer som start, stopp og endring av parametere. HMI-et inneholder ingen reguleringslogikk og tar ingen beslutninger på vegne av systemet. Det fungerer utelukkende som et grensesnitt mellom bruker og system, og forholder seg kun til de grensesnittene som er definert i grensesnittlaget.

En viktig konsekvens av denne ansvarsdelingen er at hver del av systemet kan endres, testes og videreutvikles isolert. Ny hardware kan tas i bruk uten å endre reguleringslogikken, reguleringsstrategien kan forbedres uten å endre HMI-et, og brukergrensesnittet kan videreutvikles uten å påvirke hvordan systemet faktisk styres. Dette er et sentralt prinsipp både i profesjonell automasjon og i moderne programvareutvikling.

For en automasjonsstudent er denne ansvarsdelingen avgjørende å forstå før man gjør endringer i systemet. Ved å respektere grensene mellom lagene unngår man å blande roller, noe som ofte fører til uoversiktlige og feilutsatte løsninger. Balltrack er derfor ikke bare et fungerende reguleringsystem, men også et bevisst eksempel på god struktur og disiplinert systemdesign.

1.4 Reguleringsprinsipper og systemflyt

Reguleringsprinsippet i Balltrack følger samme grunnleggende tankegang som benyttes i klassiske, lukkede reguleringsløyper innen industriell automasjon. Systemet arbeider kontinuerlig i en syklisk struktur der måling, beregning og påvirkning av prosessen gjentas med et fast tidsintervall. Denne sykliske utførelsen kan direkte sammenlignes med en OB1-syklus i et PLS-system.

I hver syklus starter systemet med å lese prosessverdien. Denne verdien representerer ballens faktiske posisjon på banen og er resultatet av en analog måling som er konvertert til en digital verdi. Prosessverdien er normalisert til et intervall mellom 0 og 1 for å gi en enhetlig og skalerbar representasjon av posisjon, uavhengig av sensoroppløsning eller fysisk utforming av banen.

Deretter beregnes avviket mellom settpunktet og prosessverdien. Settpunktet representerer den ønskede ballposisjonen og kan endres av brukeren via HMI-et. Avviket er den sentrale inngangsverdien til regulatoren og uttrykker hvor langt systemet er fra ønsket tilstand. Et positivt eller negativt avvik indikerer både retning og størrelse på reguleringsbehovet.

Avviket mates videre inn i PID-regulatorene. Regulatoren beregner et styresignal basert på proporsjonal-, integral- og derivasjonsledd. Det proporsjonale leddet reagerer direkte på avvikets størrelse, det integrerende leddet tar hensyn til akkumulert avvik over tid, og det deriverende leddet reagerer på endringer i avviket. I Balltrack er disse leddene tilgjengelige og synlige for brukeren, slik at regulatorens virkemåte kan studeres i praksis.

Regulatorens utgang er et normalisert styresignal som representerer hvor mye systemet skal påvirke prosessen i neste øyeblikk. Dette signalet begrenses til definerte minimums- og maksimumsverdier for å sikre stabil og trygg drift. Deretter oversettes styresignalet til et fysisk signal i form av en pulsbreddemodulert utgang som styrer servoens posisjon. Servoens bevegelse endrer den mekaniske tilstanden i systemet, noe som igjen påvirker ballens posisjon.

Parallelt med selve reguleringen oppdaterer systemet sin interne tilstand og statusinformasjon. Verdier som prosessverdi, settpunkt, regulatorutgang og regulatorledd lagres og gjøres tilgjengelige for HMI-et. Dette gjør det mulig å vise både nåværende tilstand og historiske trender i sanntid, og gir brukeren innsikt i hvordan reguleringsløyfen oppfører seg over tid.

Denne sekvensen gjentas kontinuerlig med et fast tidsintervall så lenge systemet er i aktiv drift. Tidsintervallet er valgt slik at det gir tilstrekkelig rask respons til å kontrollere prosessen, samtidig som systemet forblir stabilt og forutsigbart. Fra et automasjonsperspektiv er dette direkte analogt med en fast syklustid i en PLS, der alle beregninger utføres deterministisk innenfor hver syklus.

Samlet sett illustrerer reguleringsprinsippet og systemflyten i Balltrack hvordan et klassisk reguleringsproblem kan implementeres på en strukturert og oversiktlig måte. Systemet gjør det mulig å observere sammenhengen mellom måling, beregning og påvirkning i praksis, og gir et tydelig bilde av hvordan en lukket reguleringsløyfe fungerer, både teoretisk og praktisk.

1.5 Grensesnitt og API-er

For at et system som Balltrack skal være forståelig, robust og videreutviklingsbart, er det avgjørende at kommunikasjonen mellom systemets deler skjer gjennom tydelige og avtalte grensesnitt. I dette prosjektet er disse grensesnittene implementert som API-er, et begrep som i praksis tilsvarer faste kontrakter mellom funksjonsblokker, datablokker og HMI i klassisk PLS-programmering.

Et API definerer hvilke funksjoner som er tilgjengelige, hvilke data som kan leses og skrives, og hvordan disse kan brukes. I Balltrack betyr dette at hver del av systemet kun kjenner til de funksjonene og verdiene den faktisk trenger, og ikke til interne detaljer i andre deler. Denne avgrensningen er bevisst og danner grunnlaget for systemets modulære oppbygning.

Kontrolleren eksponerer et klart definert API som gir tilgang til sentrale operasjoner som start og stopp av reguleringen, endring av settpunkt, justering av regulatorparametere og uthenting av systemstatus. Dette API-et fungerer som kontrollerens offentlige grensesnitt og er den eneste måten andre deler av systemet kan påvirke eller hente informasjon fra reguleringslogikken. Kontrollerens interne beregninger, tilstandsvariabler og hardwarekoblinger er skjult bak dette grensesnittet.

Mellom kontrolleren og brukergrensesnittet benyttes et eget grensesnittlag som samler og strukturerer denne kommunikasjonen. Dette laget sørger for at HMI-et ikke trenger direkte kjennskap til kontrollerens interne struktur, men kan forholde seg til et stabilt sett med kommandoer og verdier. Denne løsningen tilsvarer bruken av datablokker eller strukturerte variabler som utveksles mellom PLS og HMI i industrielle systemer.

Også hardware-laget eksponerer sine funksjoner gjennom tydelige grensesnitt. Sensor- og aktuator-modulene tilbyr enkle operasjoner for lesing og skriving av signaler, uten å blande inn reguleringslogikk eller systemtilstand. Kontrolleren benytter disse grensesnittene for å hente måleverdier og sende styresignaler, men kjenner ikke til detaljer om hvordan disse operasjonene er implementert på lavt nivå.

En viktig konsekvens av denne API-baserte tilnærmingen er at systemets deler kan byttes ut eller endres uavhengig av hverandre. Ny hardware kan tas i bruk ved å endre hardware-laget, så lenge grensesnittet mot kontrolleren opprettholdes. Reguleringsalgoritmen kan forbedres eller erstattes uten at HMI-et må endres, så lenge kontrollerens API forblir det samme. Dette gir en fleksibilitet som er helt avgjørende i både undervisnings- og utviklingssammenheng.

For automasjonsstudenter er forståelsen av disse grensesnittene spesielt viktig. Det er her man lærer å skille mellom hva som er systemets funksjon, og hvordan denne funksjonen er implementert. Ved å respektere API-ene som kontrakter unngår man å blande roller og ansvar, og man legger grunnlaget for systemer som er oversiktlige, testbare og enkle å vedlikeholde.

I Balltrack er API-ene derfor ikke bare et teknisk virkemiddel, men en sentral del av systemets pedagogiske verdi. De illustrerer hvordan komplekse systemer kan bygges opp av enkle, veldefinerte deler som samarbeider gjennom klare og stabile grensesnitt.

1.6 HMI og brukergrensesnitt

Brukergrensesnittet i Balltrack er utformet for å gi en tydelig og informativ presentasjon av systemets tilstand, samtidig som det gir brukeren kontroll over de viktigste funksjonene i reguleringssystemet. HMI-et er ikke ment å skjule systemets indre virkemåte, men snarere å gjøre denne synlig og forståelig, særlig i en læringskontekst. Utformingen følger samme grunnleggende prinsipper som industrielle HMI-løsninger, der oversikt, forutsigbarhet og tydelig ansvarsdeling er sentrale krav.

HMI-et er bygget opp av flere selvstendige visuelle komponenter, ofte omtalt som widgets eller faceplates. Hver av disse representerer en avgrenset del av systemets funksjonalitet, for eksempel start og stopp av reguleringen, justering av settpunkt, endring av regulatorparametere, visning av trender eller presentasjon av systemstatus. Ved å dele brukergrensesnittet inn i slike komponenter blir både oppbygning og videreutvikling mer oversiktlig.

En viktig egenskap ved HMI-et er at det ikke inneholder noen reguleringslogikk. Alle beslutninger knyttet til hvordan systemet oppfører seg tas i kontroller- og reguleringslaget. HMI-et fungerer utelukkende som et grensesnitt der brukerhandlinger oversettes til kommandoer, og der systemets tilstand presenteres i et forståelig format. Dette tilsvarer rollen til et operatørpanel i et PLS-basert anlegg, der panelet aldri styrer prosessen direkte, men formidler operatørens intensjoner til styringssystemet.

Kommunikasjonen mellom HMI-et og kontrolleren skjer gjennom definerte grensesnitt. HMI-komponentene forholder seg til et begrenset sett med funksjoner og verdier som er gjort tilgjengelige for dem, og har ingen direkte tilgang til kontrollerens interne variabler. Dette gir et stabilt og forutsigbart samspill mellom brukergrensesnitt og styringslogikk, og gjør det mulig å endre eller utvide HMI-et uten å påvirke systemets funksjonelle kjerne.

HMI-et er også utformet for å gi kontinuerlig tilbakemelding til brukeren. Prosessverdi, settpunkt, regulatorutgang og regulatorledd vises både numerisk og grafisk, slik at brukeren kan observere hvordan systemet responderer over tid. Dette gir innsikt i dynamikken i reguleringssløyfen og gjør det mulig å se direkte konsekvenser av endringer i parametere eller settpunkt. Slik fungerer HMI-et både som et styringsverktøy og som et pedagogisk hjelpemiddel.

Ved å bruke en modulær oppbygning og tydelige grensesnitt fungerer HMI-et i Balltrack som et godt eksempel på hvordan moderne brukergrensesnitt kan bygges rundt et styringssystem uten å bryte med grunnleggende automasjonsprinsipper. For studenter gir dette en realistisk forståelse av hvordan operatørgrensesnitt samhandler med styringslogikk i industrielle anlegg, samtidig som systemets indre struktur forblir ryddig og oversiktlig.

1.7 Simulering og testing uten hardware

En sentral egenskap ved Balltrack er at systemet kan kjøres, testes og videreutvikles uten tilgang til fysisk hardware. Dette er et bevisst arkitektonisk valg som gir stor fleksibilitet i både undervisning og utvikling, og som tilsvarer bruk av simuleringsverktøy som PLCSIM eller virtual commissioning i industriell automasjon.

I stedet for å knytte hele systemet direkte til fysiske sensorer og aktuatorer, er det etablert et klart skille mellom prosesslogikk og hardware. Når systemet kjøres i simulert modus, erstattes den fysiske prosessen av en programvarebasert modell som leverer realistiske prosessverdier til kontrolleren. Kontrolleren, grensesnittlaget og HMI-et fungerer identisk uavhengig av om systemet kjører med ekte eller simulert prosess.

Fra brukerens perspektiv oppfører systemet seg på samme måte i begge tilfeller. Settpunkt kan endres, reguleringen kan startes og stoppes, og systemets respons kan observeres i sanntid gjennom brukergrensesnittet. Dette gjør det mulig å teste reguleringsstrategier, justere PID-parametere og studere systemdynamikk uten risiko for skade på hardware eller behov for fysisk oppsett.

Simulering spiller også en viktig rolle i feilsøking og videreutvikling. Ved å kunne kjøre systemet uten hardware kan utvikleren fokusere på programvarelogikk, struktur og grensesnitt uten å måtte ta hensyn til sensorstøy, mekaniske toleranser eller elektriske feil.

Når systemet senere tas i bruk med fysisk hardware, er de fleste logiske og strukturelle feil allerede avdekket.

For automasjonsstudenter er dette spesielt verdifullt. Simulert drift gjør det mulig å arbeide med prosjektet på egen maskin, uavhengig av tilgjengelig laboratorieutstyr. Det senker terskelen for å eksperimentere og gir rom for å gjøre endringer og lære av konsekvensene, uten fare for å skade anlegget eller påvirke andre brukere.

Det er viktig å understreke at simulering i Balltrack ikke er ment som en erstatning for testing med fysisk hardware, men som et supplement. På samme måte som i industrielle prosjekter brukes simulering til utvikling, verifikasjon og opplæring, før systemet testes og justeres mot den virkelige prosessen. Denne arbeidsformen bidrar til bedre kvalitet, høyere forståelse og mer effektiv utvikling.

Samlet sett gjør støtten for simulering og testing uten hardware Balltrack til et fleksibelt og pedagogisk sterkt system. Det gir en realistisk arbeidsflyt som speiler praksis i industriell automasjon, samtidig som det gir studenter og utviklere frihet til å arbeide med systemet uavhengig av fysisk oppsett.

1.8 Systemets driftsmodi og tilstander

Balltrack er ikke kun et kontinuerlig reguleringsystem, men et system som opererer i ulike driftsmodi og tilstander avhengig av brukerhandlinger, konfigurasjon og tilgjengelig hardware. Å forstå disse tilstandene er avgjørende for å kunne bruke systemet korrekt, feilsøke uønsket oppførsel og videreutvikle løsningen på en kontrollert måte.

I utgangspunktet befinner systemet seg i en inaktiv tilstand. I denne tilstanden er reguleringen stoppet, og det sendes ingen aktive styresignaler til servoen. Systemet kan likevel være fullt operativt fra et brukergrensesnittsperspektiv, med tilgang til visning av måleverdier, justering av settpunkt og endring av regulatorparametere. Denne tilstanden tilsvarer et PLS-system som er i STOP, men der HMI fortsatt er tilgjengelig for konfigurasjon og overvåking.

Når brukeren starter reguleringen via HMI-et, går systemet over i aktiv reguleringsmodus. I denne tilstanden kjøres reguleringsløyfen kontinuerlig med fast syklustid. Prosessverdien leses, avvik beregnes, regulatoren oppdateres og styresignalet sendes til aktuatoren i hver syklus. Systemet oppfører seg deterministisk og følger de prinsippene som er beskrevet i

kapittel om reguleringsprinsipp og systemflyt. Denne tilstanden tilsvarer RUN-modus i et PLS-system.

Systemet kan også operere i en form for manuell eller halvautomatisk modus, der reguleringen er deaktivert, men der enkelte utganger eller funksjoner kan styres direkte. Dette kan for eksempel være aktuelt under testing, kalibrering eller feilsøking. I slike tilfeller er det viktig at reguleringsløyfen ikke samtidig forsøker å påvirke prosessen, for å unngå motstridende kommandoer. Balltrack er derfor bygget slik at regulering og manuell styring er tydelig adskilt.

Et annet viktig skille i systemets driftsmodi er forskjellen mellom simulert drift og drift med fysisk hardware. I simulert modus genereres prosessverdier av en programvaremodell, og styresignaler påvirker kun den interne simuleringen. I hardware-modus er systemet koblet til faktiske sensorer og aktuatorer. Fra kontrollerens og HMI-ets perspektiv er disse to modusene likeverdige, men konsekvensene av handlinger er naturlig nok ulike. Dette skillet er viktig å ha bevisst forhold til under utvikling og testing.

Systemets tilstand reflekteres kontinuerlig i statusinformasjonen som gjøres tilgjengelig for HMI-et. Informasjon om gjeldende modus, siste måleverdier og regulatorutgang vises for brukeren, slik at det til enhver tid er mulig å se hvordan systemet opererer. Dette gir både økt sikkerhet og bedre forståelse av systemets oppførsel.

For en automasjonsstudent er det viktig å se disse driftsmodiene og tilstandene som en integrert del av systemdesignet, ikke som tillegg. I industrielle automasjonssystemer er tydelig håndtering av tilstander helt avgjørende for sikker og forutsigbar drift. Balltrack er bygget med samme tankegang, og gir dermed et realistisk bilde av hvordan profesjonelle styringssystemer er strukturert og operert.

1.9 Sammenheng med klassisk PLS-arkitektur

Selv om Balltrack er implementert i Python og kjøres på en generell dataplattform, er systemets grunnleggende arkitektur direkte sammenlignbar med klassisk PLS-baserte styringssystemer brukt i industriell automasjon. De samme konseptene som benyttes i PLS-programmering som syklisk kjøring, tydelig IO-struktur, databaserte grensesnitt og adskilt HMI er gjennomgående i prosjektets oppbygning.

Kontrolleren i Balltrack tilsvarer funksjonelt en PLS-CPU. Den utfører all styrings- og reguleringslogikk i en syklisk oppdatering, på samme måte som et OB1-program i Siemens TIA Portal eller tilsvarende hovedsyklus i andre PLS-miljøer. Innenfor hver syklus leses

innganger, logikk utføres og utganger oppdateres deterministisk. Denne strukturen gir forutsigbarhet og gjør systemets oppførsel lett å analysere og feilsøke.

Hardware-laget i Balltrack kan direkte sammenlignes med IO-moduler i et PLS-system. Sensorlesing via analog-til-digital-omformer tilsvarer analoge innganger, mens servoaktivering via PWM tilsvarer analoge eller raske digitale utganger. Kontrolleren forholder seg til disse gjennom definerte grensesnitt, på samme måte som et PLS-program forholder seg til adresserte IO-signaler uten å kjenne detaljene i den elektriske implementasjonen.

Grensesnittlaget mellom kontroller og HMI tilsvarer datablocker eller strukturerte variabler som eksponeres mot et operatørpanel. I stedet for at HMlen leser og skriver direkte i kontrollerens interne variabler, skjer all kommunikasjon gjennom et kontrollert sett med tilgjengelige verdier og kommandoer. Dette er i tråd med god praksis i industrielle systemer, der HMI-et kun skal ha tilgang til det som er nødvendig for operatørbetjening og overvåking.

HMI-et i Balltrack tilsvarer et operatørpanel eller SCADA-system i et industrielt anlegg. Det presenterer prosessverdier, settpunkt, regulatorutgang og systemstatus, og gir operatøren mulighet til å starte og stoppe reguleringen samt justere relevante parametere. HMI-et tar ingen beslutninger på egen hånd, men fungerer som et brukergrensesnitt mot styringssystemet, akkurat som i en PLS-basert løsning.

Støtten for simulering i Balltrack kan sammenlignes med bruk av PLCSIM eller virtual commissioning i industriell automasjon. Ved å kunne kjøre samme styringslogikk mot en simulert prosess oppnås en utviklings- og testflyt som speiler profesjonell praksis. Dette gjør systemet særlig egnet som læringsplattform, fordi studentene kan jobbe med realistiske strukturer uten å være avhengige av fysisk utstyr til enhver tid.

Samlet sett viser Balltrack hvordan prinsippene fra klassisk PLS-arkitektur kan overføres til en moderne programvareplattform uten å miste struktur, disiplin eller forståelighet. For automasjonsstudenter gir dette et tydelig bindeledd mellom teori, tradisjonelle verktøy og moderne implementasjonsformer, og bidrar til å gjøre overgangen mellom skole og industri mer naturlig.

1.10 Verktøy for utvikling og testing

For å kunne utvikle, teste og videreføre Balltrack er det nødvendig å forstå hvilke verktøy som benyttes i prosjektet og hvordan disse inngår i arbeidsflyten. Disse verktøyene er ikke en del av selve reguleringssystemet i drift, men utgjør grunnlaget for all utvikling, feilsøking og videreutvikling. En grunnleggende forståelse av verktøyene er derfor en forutsetning for å kunne arbeide effektivt med prosjektet.

1.10.1 Versjonskontroll med Git og GitHub

Prosjektet Balltrack er versjonskontrollert ved hjelp av Git og lagret i et GitHub-repositorium. Git brukes for å holde oversikt over endringer i kildekoden over tid, slik at utviklingen kan spores, feil kan reverseres og flere personer kan arbeide med samme kodebase uten å overskrive hverandres arbeid. GitHub fungerer som en sentral lagringsplass for prosjektet og gjør det mulig å klonе prosjektet til nye maskiner.

For en student som skal komme i gang med prosjektet innebærer dette normalt å klonе repositoriet til egen maskin. Videre arbeid skjer lokalt, og endringer kan lagres som commits før de eventuelt lastes opp igjen. I denne sammenhengen brukes Git primært som et verktøy for historikk, sikkerhet og struktur, ikke som et avansert samhandlingsverktøy med komplekse arbeidsflyter.

1.10.2 Python-miljø og avhengigheter

Balltrack er utviklet i Python og forutsetter at et fungerende Python-miljø er installert på utviklingsmaskinen. Prosjektet benytter eksterne Python-pakker for blant annet grafikk, kommunikasjon og hardware-tilgang. For å unngå konflikter med andre Python-prosjekter anbefales det å bruke et virtuelt Python-miljø der alle nødvendige avhengigheter installeres isolert.

Ved å samle prosjektets avhengigheter i et eget miljø sikres det at riktig versjon av bibliotekene benyttes, og at prosjektet kan kjøres på ulike maskiner med samme oppførsel. Dette er en etablert praksis i profesjonell programvareutvikling og gir en forutsigbar utviklingsopplevelse.

1.10.3 Utviklingsmiljø (VS Code)

Visual Studio Code er anbefalt utviklingsmiljø for Balltrack. VS Code gir god støtte for Python-utvikling, integrert terminal, Git-integrasjon og utvidelser som gjør arbeidet mer oversiktlig. For dette prosjektet brukes VS Code hovedsakelig som editor, kjøreplattform og navigasjonsverktøy i kodebasen.

Ved å åpne prosjektmappen direkte i VS Code får utvikleren tilgang til hele filstrukturen, noe som gjør det enkelt å orientere seg mellom hardware-moduler, kontroller, grensesnitt

og GUI-komponenter. Integrasjonen med Git gjør det også enkelt å se hvilke filer som er endret og å lagre endringer på en kontrollert måte.

1.10.4 Terminalbruk på Windows og Linux

Terminalen er et sentralt verktøy i arbeidet med Balltrack, både under utvikling og testing. Terminalen brukes blant annet til å opprette virtuelle Python-miljøer, installere nødvendige pakker, starte programmet og kjøre tester. Selv om grafiske verktøy benyttes der det er hensiktsmessig, gir terminalen en direkte og presis måte å samhandle med systemet på.

Prosjektet kan utvikles både på Windows og Linux. På Windows benyttes typisk PowerShell eller Command Prompt, mens Linux-systemer bruker Bash eller tilsvarende skall.

Kommandoene og arbeidsflyten er i stor grad like på tvers av plattformene, og erfaring med en plattform er derfor lett overførbart til den andre. I praksis utvikles og testes systemet ofte på Windows eller Linux-maskiner, mens endelig kjøring mot fysisk hardware skjer på et Linux-basert system, for eksempel en Raspberry Pi.

Ved å bruke terminalen aktivt får studenten innsikt i hvordan programmet faktisk startes, hvordan miljøet settes opp, og hvordan feil kan identifiseres gjennom meldinger og utskrifter. Dette er en viktig del av å arbeide profesjonelt med både automasjon og programvareutvikling.

1.11 Hvordan orientere seg i prosjektet

Når en ny student eller utvikler overtar Balltrack, er det viktig å ha en strukturert tilnærming til hvordan prosjektet leses og forstås. Systemet er bevisst bygget modulært, og det er derfor verken nødvendig eller hensiktsmessig å starte med å lese all kildekode i detalj. I stedet bør prosjektet forstås ovenfra og ned, med fokus på helhet før detaljer.

Det naturlige startpunktet er å forstå hvordan systemet kjøres og oppleves som bruker. Ved å starte programmet i simulert modus og observere HMI-et får man raskt et bilde av hvilke funksjoner systemet har, hvilke signaler som vises, og hvordan reguleringen oppfører seg. Dette gir kontekst til resten av prosjektet og gjør det lettere å forstå hvorfor de ulike modulene eksisterer.

Når den overordnede funksjonen er forstått, bør oppmerksomheten rettes mot kontroller- og reguleringslaget. Dette er kjernen i systemet og tilsvarer PLS-programmet i et industrielt

anlegg. Her finnes reguleringsløyfen, tilstandshåndtering og koblingen mellom måling og aktivering. Ved å lese kontrolleren med fokus på flyt og ansvar, fremfor detaljer i hver beregning, får man innsikt i hvordan systemet er ment å fungere.

Deretter er det naturlig å se på grensesnittlaget mellom kontroller og HMI. Dette laget definerer hvilke funksjoner og verdier som er tilgjengelige for brukergrensesnittet, og fungerer som en kontrakt som resten av systemet forholder seg til. For en ny utvikler er dette et viktig referansepunkt, fordi det viser hvilke deler av systemet som er ment å brukes eksternt, og hvilke som er interne detaljer.

Hardware-modulene bør leses med forståelsen av at de representerer lavnivåfunksjonalitet. Disse modulene er ofte svært konkrete og tekniske, og det er sjelden nødvendig å gjøre endringer her med mindre man skal bytte sensor, aktuator eller fysisk oppsett. For de fleste videreutviklinger er det tilstrekkelig å vite hvilke grensesnitt hardware-laget tilbyr, ikke hvordan hver enkelt operasjon er implementert.

HMI- og GUI-modulene kan studeres når man ønsker å endre eller utvide brukergrensesnittet. Her er strukturen bygget rundt selvstendige widgets, som hver har et klart ansvar. Ved å forstå denne oppdelingen blir det mulig å legge til nye visninger eller kontrollfunksjoner uten å påvirke resten av systemet.

En viktig rettesnor ved orientering i prosjektet er å respektere arkitekturen slik den er beskrevet i denne rapporten. Endringer bør alltid gjøres i det laget de naturlig hører hjemme i. Regulering endres i kontrolleren, ikke i HMI-et. Nye visninger legges til i GUI-et, ikke i reguleringslogikken. Ved å følge denne disiplinen unngår man å introdusere uoversiktlige avhengigheter og gjør prosjektet lettere å vedlikeholde over tid.

For en automasjonsstudent er dette også en verdifull øvelse i profesjonell arbeidsmetodikk. Å kunne orientere seg i et eksisterende system, forstå arkitekturen før man endrer noe, og gjøre målrettede inngrep på riktig sted, er ferdigheter som er direkte overførbare til arbeid med industrielle automasjonssystemer.

- V8_BALLTRACK/
 - main.py
 - sitecustomize.py
 - structure.txt
 - config/
 - settings.py
 - controller/
 - dummy_controller.py
 - pid.py
 - position_controller.py
 - hardware/
 - adc/
 - ads1015.py
 - pwm/
 - pca9685.py
 - gui/
 - gui_tags.py
 - app_screen.py
 - widgets/
 - base_widget.py
 - registry.py
 - control/
 - control_widget.py
 - servo_widget.py
 - setpoint_widget.py
 - monitoring/
 - balltrack_visual_widget.py
 - footer_widget.py
 - log_widget.py
 - plot_widget.py
 - pid/
 - pid_widget.py
 - tests/
 - mock_gui_tags.py
 - preview_widgets.py
 - test_ads1015.py
 - test_pwm.py
 - test_pid.py
 - test_position_controller.py
 - test_gui.py
 - tools/
 - grovehat_adc_fallback.py

1.12 Hvordan jobbe videre med prosjektet

Når Balltrack er forstått som system og arkitektur, er neste steg å kunne videreutvikle prosjektet på en strukturert og trygg måte. Målet med videre arbeid bør ikke være å gjøre raske endringer, men å bygge videre på de prinsippene systemet allerede er basert på. Dette gir både bedre teknisk kvalitet og en mer profesjonell arbeidsmetodikk.

Videreutvikling bør alltid starte med en klar forståelse av hva som ønskes endret eller lagt til. Det kan være justering av reguleringsstrategi, forbedring av brukergrensesnittet, støtte for ny hardware eller utvidelse av visualisering og logging. Før selve implementasjonen bør man plassere endringen arkitektonisk: hører den hjemme i hardware-laget, i kontrolleren, i grensesnittlaget eller i HMI-et. Dette steget er avgjørende for å unngå at ansvar blandes.

Reguleringsrelaterte endringer, som justering av PID-struktur, begrensninger eller samplingsstrategi, skal alltid gjøres i kontroll- og reguleringslaget. HMI-et skal ikke inneholde reguleringslogikk, og hardware-laget skal ikke ta beslutninger om systemets oppførsel. Ved å holde reguleringen samlet i kontrolleren bevares systemets oversiktighet og forutsigbarhet.

Endringer i brukergrensesnittet bør gjøres ved å legge til eller justere widgets, uten å endre hvordan kontrolleren fungerer internt. Dersom ny informasjon ønskes vist i HMI-et, bør denne eksponeres gjennom grensesnittlaget på en kontrollert måte. På denne måten forblir HMI-et løst koblet til styringslogikken, slik arkitekturen forutsetter.

Når ny hardware skal tas i bruk, bør dette skje ved å utvide eller erstatte hardware-modulene, uten å endre kontrollerens eksterne grensesnitt. Så lenge hardware-laget tilbyr samme funksjonalitet mot kontrolleren, kan resten av systemet fungere uendret. Dette er samme prinsipp som benyttes i industrielle systemer når IO-moduler byttes eller prosessutstyr oppgraderes.

Testing bør alltid starte i simulert modus. Simulering gir rask feedback og reduserer risiko for feil som kan føre til ustabil regulering eller skade på fysisk utstyr. Først når endringer fungerer som forventet i simulering, bør de testes mot fysisk hardware. Denne arbeidsformen gir bedre kontroll og speiler profesjonell praksis innen automasjon.

Det anbefales også å bruke Git aktivt under videreutvikling. Små, hyppige endringer med tydelige commit-meldinger gjør det enklere å forstå utviklingshistorikken og å gå tilbake dersom noe ikke fungerer som ønsket. Dette er spesielt viktig i et studentprosjekt der flere personer kan bidra over tid.

Til slutt er det viktig å se Balltrack som en læringsplattform, ikke et ferdig produkt. Systemet er ment å utforskes, justeres og forbedres. Samtidig gir den eksisterende arkitekturen et rammeverk som bør respekteres. Ved å arbeide innenfor disse rammene kan prosjektet videreutvikles på en måte som både gir faglig utbytte og erfaring med profesjonell systemutvikling.

1.13 Begrensninger og forutsetninger

Balltrack er utviklet som et lærings- og demonstrasjonssystem innen automasjon og regulering. Selv om systemet er strukturert etter prinsipper som brukes i industriell automasjon, er det viktig å være bevisst på hvilke begrensninger og forutsetninger prosjektet bygger på. En tydelig forståelse av disse rammene er avgjørende for korrekt bruk, videreutvikling og vurdering av systemets egnethet i ulike sammenhenger.

En grunnleggende forutsetning i prosjektet er at systemet benyttes i en kontrollert lærings- eller utviklingskontekst. Balltrack er ikke utviklet som et ferdig industriprodukt og inneholder ikke funksjoner for sikkerhetskritisk drift. Det er for eksempel ikke implementert redundans, feilsikring på maskinvare-nivå eller sertifiserte sikkerhetsmekanismer slik man finner i industrielle styringssystemer som skal brukes i produksjonsmiljøer.

Reguleringsløsningen er bevisst holdt oversiktlig og pedagogisk. PID-regulatoren er implementert for å være lett å forstå og justere, ikke for å dekke alle spesialtilfeller eller avanserte reguleringsstrategier. Systemet forutsetter at brukeren har grunnleggende forståelse for reguleringsteori og er i stand til å tolke konsekvensene av endringer i regulatorparametere. Feilinnstillinger kan føre til ustabil oppførsel, spesielt ved bruk av fysisk hardware.

Når systemet benyttes med fysisk hardware, forutsettes det at oppsettet er korrekt montert og kalibrert. Sensorer og aktuatorer må være riktig tilkoblet, og de mekaniske komponentene må være i forsvarlig stand. Balltrack inneholder ikke omfattende selvdiagnostikk for å oppdage feil i det fysiske oppsettet, og det er derfor brukerens ansvar å sikre at hardware fungerer som forventet før reguleringen aktiveres.

Prosjektet forutsetter også et bestemt teknologisk rammeverk. Systemet er utviklet i Python og benytter spesifikke biblioteker for grafikk, kommunikasjon og hardware-tilgang. Endringer i underliggende plattform, operativsystem eller biblioteksversjoner kan påvirke systemets oppførsel. Det anbefales derfor å benytte de verktøyene og arbeidsmetodene som er beskrevet i kapittelet om utviklings- og testverktøy.

En annen viktig begrensning er at systemet er tilpasset en bestemt type prosess. Balltrack demonstrerer prinsipper for regulering av en enkel, en-dimensjonal prosess. Selv om

arkitekturen er generell og kan overføres til andre anvendelser, vil selve reguleringslogikken og prosessmodellen måtte tilpasses dersom systemet brukes til å styre andre typer prosesser.

Ved videreutvikling er det også viktig å være bevisst på arkitekturen som systemet bygger på. Prosjektet forutsetter at endringer gjøres i riktig lag og at grensesnittene mellom lagene respekteres. Dersom disse prinsippene brytes, kan systemet raskt bli vanskelig å forstå og vedlikeholde, noe som reduserer både pedagogisk verdi og teknisk kvalitet.

Samlet sett er Balltrack best egnet som et verktøy for læring, eksperimentering og demonstrasjon av automasjonsprinsipper. Ved å være bevisst systemets begrensninger og forutsetninger kan prosjektet brukes på en trygg og faglig forsvarlig måte, samtidig som det gir rom for videre utforskning og utvikling innenfor klare rammer.

1.14 Videre bruk som handover

Denne rapporten er utformet for å fungere som et handover-dokument for Balltrack, både i en akademisk og en mer teknisk sammenheng. Hensikten er at dokumentet skal gjøre det mulig for nye studenter, faglærere eller utviklere å ta over systemet uten behov for muntlig overlevering eller inngående kjennskap til prosjektets historie. Rapporten skal kunne stå på egne ben som referanse og veiledning.

Som handover-dokument er rapporten bygget opp slik at leseren først får en helhetlig forståelse av systemets funksjon og arkitektur, før detaljnivået gradvis økes. Dette gjør det mulig å bruke dokumentet selektivt. En ny bruker kan lese de innledende kapitlene for å forstå systemets formål og oppbygning, mens en utvikler som skal gjøre endringer kan fokusere på kapitlene om grensesnitt, verktøy og videre arbeid.

Dokumentet er også ment å være levende. Etter hvert som Balltrack videreutvikles, kan rapporten oppdateres for å reflektere nye funksjoner, endringer i arkitektur eller justerte arbeidsmetoder. Ved å holde rapporten oppdatert sammen med kildekoden bevares sammenhengen mellom system og dokumentasjon, noe som er viktig for langsiktig bruk og vedlikehold.

I undervisningssammenheng kan rapporten brukes som støtte for introduksjon til prosjektet, som referanse under arbeid, og som grunnlag for refleksjon rundt arkitekturvalg og reguleringsprinsipper. Ved prosjektarbeid kan dokumentet fungere som en felles forståelsesplattform for grupper som samarbeider om videreutvikling, og bidra til at alle arbeider etter samme strukturelle prinsipper.

I en mer teknisk eller profesjonell kontekst kan rapporten fungere som en første teknisk gjennomgang av systemet. Den gir oversikt over arkitektur, ansvarsdeling og grensesnitt, og gjør det mulig å vurdere systemets egnethet som demonstrator, prototype eller utgangspunkt for videre utvikling. Samtidig er det tydelig beskrevet hvilke begrensninger systemet har, slik at forventninger kan justeres deretter.

Avslutningsvis er det viktig å understreke at Balltrack ikke er ment å være et ferdig produkt, men et strukturert og gjennomtenkt utgangspunkt. Rapporten er derfor skrevet med fokus på forståelse, overtakelse og videreføring, fremfor fullstendig spesifisering. Ved å bruke dokumentet aktivt og respektere de arkitektoniske prinsippene som er beskrevet, kan systemet videreutvikles på en måte som gir både faglig verdi og praktisk erfaring med profesjonell automasjon.

1.15 Ordliste

API (Application Programming Interface)

Et definert grensesnitt som beskriver hvilke funksjoner og data som er tilgjengelige mellom ulike deler av et system. I Balltrack brukes API-er som kontrakter mellom hardware, kontroller og HMI, tilsvarende grensesnitt mellom funksjonsblokker og datablokker i PLS-systemer.

Balltrack

Den fysiske prosessen der en ball beveger seg langs en bane, og som styres ved hjelp av en servoaktuert mekanisme. Ballens posisjon utgjør systemets prosessverdi.

Controller (Kontroller)

Den delen av systemet som inneholder all styrings- og reguleringslogikk. Kontrolleren tilsvarer en PLS-CPU og er ansvarlig for reguleringsløyfen, tilstandshåndtering og kommunikasjon med hardware og HMI.

Faceplate / Widget

En selvstendig grafisk komponent i HMI-et som representerer en avgrenset funksjon, for eksempel settpunktstyring, PID-innstillinger eller trendvisning. Tilsvarer faceplates brukt i industrielle HMI-systemer.

GUI / HMI (Human Machine Interface)

Brukergransnittet som gjør det mulig for operatøren å overvåke systemets tilstand og sende kommandoer til kontrolleren. HMI-et inneholder ingen reguleringslogikk.

Hardware-lag

Den delen av systemet som håndterer direkte kommunikasjon med fysiske sensorer og aktuatorer, som ADC og PWM-kontroller. Hardware-laget inneholder ingen styringslogikk.

Normalisering

Prosessen med å skalere fysiske verdier til et felles intervall, ofte mellom 0 og 1. I Balltrack brukes normaliserte verdier for posisjon, settpunkt og regulatorutgang for å gjøre systemet enklere å forstå og tilpasse.

OB1 / Syklisk kjøring

Begrep fra PLS-programmering som beskriver hovedprogrammet som kjøres kontinuerlig i faste sykluser. Balltrack benytter en tilsvarende syklisk oppdatering i kontrolleren.

PLS (Programmerbar Logisk Styring)

Industriell styreenhet brukt til automatisering av prosesser. Prinsippene for struktur, IO-håndtering og HMI-kommunikasjon i Balltrack er direkte inspirert av PLS-baserte systemer.

Prosessverdi (PV)

Den målte verdien fra prosessen, i dette tilfellet ballens faktiske posisjon på banen.

PWM (Pulse Width Modulation)

En metode for å styre effekt eller posisjon ved å variere bredden på pulser i et digitalt signal. I Balltrack brukes PWM til å styre servoens posisjon.

Reguleringsløyfe

Den kontinuerlige prosessen der systemet måler prosessverdi, sammenligner den med settpunkt, beregner et styresignal og påvirker prosessen for å redusere avviket.

Settpunkt (SP)

Den ønskede verdien for prosessverdien. I Balltrack representerer settpunktet ønsket ballposisjon.

Simulering / Virtual commissioning

Kjøring av styrings- og reguleringslogikk mot en simulert prosess i stedet for fysisk hardware. Brukes til utvikling, testing og opplæring før eller uten fysisk anlegg.

Styresignal (U)

Utgangen fra regulatoren som brukes til å påvirke prosessen. I Balltrack oversettes styresignalet til et PWM-signal som styrer servoen.

Tilstand / Driftsmodus

Systemets nåværende operasjonelle status, for eksempel stoppet, regulerende, manuell eller simulert drift. Tilstand avgjør hvordan systemet reagerer på brukerkommandoer.

